

MINISTERUL EDUCAȚIEI, CULTURII ȘI CERCETĂRII
AL REPUBLICII MOLDOVA
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică
Departamentul Inginerie Software și Automatică

Canter Daniel - FAF-242

Report

Laboratory work n.1-2

of Operating Systems

Checked by:

Reițman Patricia, university assistant

DISA, FCIM, UTM

Chișinău – 2026

1. LAB 1

1.1. Part 1 — Files and Permissions

1.1.1. File ownership

Who owns the files you just created, and which group?

The owner of a file on creation is determined by the file permissions on the parent folder. Since the files are owned by the user `debian`, group `debian`, the files also inherit that.

```
debian@localhost:~/os-lab1/demo$ ls -l note.txt # read the rwx / owner / group columns

chmod 600 note.txt # owner-only read+write
ls -l note.txt
chmod 644 note.txt
-rw-rw-r-- 1 debian debian 24 Sep 28 14:26 note.txt
-rw------- 1 debian debian 24 Sep 28 14:26 note.txt
```

1.1.2. Permission string

What do the ten characters at the start of `ls -l` mean for `note.txt` after `chmod 600`?

The first characters are deciphered by file-type / user / group / all, in triplets except the first one.

The numbers come from the bits that hold this:

Bit	Value	Meaning
`r`	4	Read
`w`	2	Write
`x`	1	Execute

And the `600` is just short to say `rw-/---/---`.

1.1.3. Root directories

Name two directories directly under `/` and say, in one line each, what they hold.

- `/tmp` holds short-time temporary files which are deleted periodically
- `/bin` holds executable binaries

1.2. Part 2 — Processes

1.2.1. PID 1

What is the PID of process number 1, and what is it (look it up)?

The PID 1 is the first process that is run by the kernel. In Linux all of the processes are children of other processes except this one. This special process is run by the kernel on boot and it's used to initialize all other programs which your computer may run. On modern systems it's usually systemd.

1.2.2. Process count

Roughly how many processes were running on your idle VM?

```
debian@localhost:~/os-lab1/demo$ ps -aux | wc -l
130
```

It's roughly 130.

1.2.3. Process state

From `/proc/<pid>/status`, what does the "State:" line say for your sleeping process?

```
Name:   sleep
Umask:  0002
State:  R (running)
Tgid:   1711
Ngid:   0
Pid:    1711
PPid:   1007
TracerPid:      0
Uid:    1000    1000    1000    1000
Gid:    1001    1001    1001    1001
FDSize: 256
Groups: 4 20 24 25 27 29 30 44 46 1000 1001
NStgid: 1711
NSpid:  1711
NSpgid: 1711
NSsid:  1007
Kthread:        0
VmPeak:     2460 kB
VmSize:     2460 kB
VmLck:         0 kB
[1]+  Terminated              sleep 300
```

The state is running.

1.3. Part 3 — Memory

```
debian@localhost:~/os-lab1/demo$ free -h
               total        used        free      shared  buff/cache   available
Mem:           3.8Gi       405Mi       3.3Gi       816Ki       316Mi       3.4Gi
Swap:             0B          0B          0B
```

1.3.1. Total and free RAM

How much total RAM does the VM have, and how much is free?

1.3.2. Swap

What is swap, and how much is configured?

Swap is memory saved on disk. In the VM there is no swap configured.

1.3.3. Resident memory of `sleep`

How much resident memory (VmRSS) does a bare `sleep` process use, and does that surprise you?

```
debian@localhost:~/os-lab1/demo$ free -h # total / used / free RAM and swap
cat /proc/meminfo | head -6
sleep 300 & PID=$!
grep VmRSS /proc/$PID/status # real RAM used by this process
kill $PID
               total        used        free      shared  buff/cache   available
Mem:           3.8Gi       401Mi       3.3Gi       816Ki       316Mi       3.4Gi
Swap:             0B          0B          0B
MemTotal:        4015360 kB
MemFree:         3497696 kB
MemAvailable:    3604192 kB
Buffers:           17424 kB
Cached:           289916 kB
SwapCached:            0 kB
[1] 1769
VmRSS:      1876 kB
[1]+  Terminated              sleep 300
```

The sleep process uses a few kB; this is mostly the code loaded on to RAM.

1.4. Part 4 — Devices and Filesystems

```
debian@localhost:~$ df -h # mounted filesystems and free space
lsblk # the block devices (disks) themselves
du -sh ~/os-lab1 # size of your work folder
ls -l /dev | head # devices presented as files
mount | head
Filesystem                                  Size  Used Avail Use% Mounted on
udev                                        1.9G     0  1.9G   0% /dev
tmpfs                                       393M  812K  392M   1% /run
/dev/sda1                                    63G  2.0G   59G   4% /
tmpfs                                       2.0G     0  2.0G   0% /dev/shm
tmpfs                                       5.0M     0  5.0M   0% /run/lock
tmpfs                                       2.0G     0  2.0G   0% /tmp
tmpfs                                       1.0M     0  1.0M   0% /run/credentials/systemd-journald.service
tmpfs                                       1.0M     0  1.0M   0% /run/credentials/systemd-resolved.service
/dev/sda15                                  124M  8.9M  115M   8% /boot/efi
tmpfs                                       1.0M     0  1.0M   0% /run/credentials/systemd-networkd.service
192.168.100.191:/mnt/pool/NFS/fast-workload  685G     0  685G   0% /mnt/nfs/data
tmpfs                                       1.0M     0  1.0M   0% /run/credentials/serial-getty@ttyS0.service
tmpfs                                       1.0M     0  1.0M   0% /run/credentials/getty@tty1.service
tmpfs                                       393M  4.0K  393M   1% /run/user/1000
NAME    MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda       8:0    0   64G  0 disk
├─sda1    8:1    0 63.9G  0 part /
├─sda14   8:14   0    3M  0 part
└─sda15   8:15   0  124M  0 part /boot/efi
sr0      11:0    1    4M  1 rom
16K     /home/debian/os-lab1
total 0
crw-r--r-- 1 root root     10, 235 Sep 28 13:05 autofs
drwxr-xr-x 2 root root         140 Sep 28 13:05 block
drwxr-xr-x 2 root root          80 Sep 28 13:05 bsg
crw------- 1 root root     10, 234 Sep 28 13:05 btrfs-control
drwxr-xr-x 3 root root          60 Sep 28 13:05 bus
lrwxrwxrwx 1 root root           3 Sep 28 13:05 cdrom -> sr0
drwxr-xr-x 2 root root        2960 Sep 28 13:05 char
crw------- 1 root tty       5,   1 Sep 28 13:05 console
lrwxrwxrwx 1 root root          11 Sep 28 13:05 core -> /proc/kcore
sysfs on /sys type sysfs (rw,nosuid,nodev,noexec,relatime)
proc on /proc type proc (rw,nosuid,nodev,noexec,relatime)
udev on /dev type devtmpfs (rw,nosuid,relatime,size=1986176k,nr_inodes=496544,mode=755,inode64)
devpts on /dev/pts type devpts (rw,nosuid,noexec,relatime,gid=5,mode=600,ptmxmode=000)
tmpfs on /run type tmpfs (rw,nosuid,nodev,noexec,relatime,size=401536k,mode=755,inode64)
/dev/sda1 on / type ext4 (rw,relatime,discard,errors=remount-ro)
securityfs on /sys/kernel/security type securityfs (rw,nosuid,nodev,noexec,relatime)
tmpfs on /dev/shm type tmpfs (rw,nosuid,nodev,inode64)
cgroup2 on /sys/fs/cgroup type cgroup2 (rw,nosuid,nodev,noexec,relatime,nsdelegate,memory_recursiveprot)
pstore on /sys/fs/pstore type pstore (rw,nosuid,nodev,noexec,relatime)
```

1.4.1. Root filesystem device

Which device is your root filesystem `/` mounted on?

The device that's mounted on `/` is `/dev/sda1`, which is the virtual hard-drive.

1.4.2. A `/dev` entry

Give one entry from `/dev` and say what real thing it stands for.

`/dev/zero` is a special file that contains all zeros.

1.4.3. Everything is a file

In one sentence: what does "everything is a file" mean, based on what you saw?

All of the connected devices, ports, etc. are files that map memory; any of them are abstracted with `files`.

2. LAB 2

2.1. Part 2 — Using xv6

2.1.1. Bundled programs

List three programs xv6 ships with.

- `cat`
- `echo`
- `wc`

2.1.2. Pipes

You just used a pipe (`|`) inside xv6. Which two OS features must exist for a pipe to work? (Hint: think processes and communication.)

Read and write should exist as a minimum.

2.1.3. xv6 shell vs. Linux shell

In one sentence: how does the xv6 shell compare to the Linux shell from Lab 1?

The shell is more minimal, I can move the cursor up and down, there are fewer features.

2.2. Part 3 — System Calls

2.2.1. System calls in `cat`

Which system calls does `user/cat.c` use, and what does each one ask the kernel to do?

The program uses read, write and pipe. It asks the kernel to read and write data to files; also pipe is a special thing to communicate between processes.

2.2.2. Location of `sys_read`

In which kernel file, and on which line, is `sys_read` implemented?

The `kernel/sysfile.c`.

2.2.3. `kernel/` vs. `user/`

In one sentence: what is the difference between the code in `kernel/` and the code in `user/`?

The code in `kernel/` is used to compile the kernel; it contains system-calls and all other OS internals and provides an API for user-space programs.

The `user/` is used to compile user-space programs like `ls` or `cat`.

2.3. Part 4 — The `sleep` Program

This is the `sleep` program for the VM:

```
#include "kernel/types.h"
#include "user/user.h"

// Sleeps 10*n ticks.
// One tick is around 100ms
int
main(int argc, char *argv[])
{
  if (argc != 2) {
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1])*10); // system call into the kernel
  exit(0);
}
```

3. CONCLUSION

In Lab 1 we worked on a Linux VM and looked at file ownership and permissions, processes and `/proc`, memory and swap, and how devices are exposed as files under `/dev`. In Lab 2 we ran xv6, used its minimal shell and programs, read how user programs such as `cat` talk to the kernel, and wrote a `sleep` program that does its work through a single system call.

The main takeaway is the split between user space and the kernel: user programs never touch the hardware directly, they ask the kernel for everything through system calls.